

Manual for: OCamlSDL2

Table of Contents

- 1: Initialisation
- 2: Drawing Points, X and Y Coordinates
- 3: Drawing Squares and Rectangles
- 4: Transparency
- 5: Drawing Lines
- 6: Filling Squares and Rectangles, And Signatures
- 7: Animation Loop, And Recursion
- 8: A Better Organisation of the Code, The "MVC" organisation
- 9: Continuing with the "MVC" model, Populating the area with several Entities
- 10: Processing Events, The Event Loop
- 11: Loading Sprites

License

You can (re)use this manual under the conditions of the GNU/FDL license, and/or cc-by-sa(2.5).

Pieces of code from this manual can be (re)used under either:

- any fsf-approved license
- any osi-approved license

fsf: free-software foundation

osi: open-source initiative

Manual for: OCamlSDL2

Initialisation

```
let () =  
  Sdl.init [`VIDEO]
```

Initialises the SDL2 library, for use with something visible on screen.

Prefixed, or Prefixless?

If you prefer to use the modules, without the 'Sdl' prefix, you can add this open, at the beginning of your file:

```
open Sdl
```

Render, and Renderers

A possibility to draw on screen, inside the window, is to use a renderer with the render module.

We can initialise a window, and a renderer, with the function:

```
Sdlrender.create_window_and_renderer ~width ~height ~flags:[]
```

Here is an example, which will draw four points in a window, with a renderer, and the render module:

```
let () =  
  Sdl.init [`VIDEO];  
  
  let width, height = (320, 240) in  
  
  let window, renderer =  
    Sdlrender.create_window_and_renderer ~width ~height ~flags:[]  
  in  
  
  Sdlrender.set_draw_color renderer ~rgb:(0,255,255) ~a:255;  
  Sdlrender.clear renderer;  
  
  Sdlrender.set_draw_color renderer ~rgb:(0,0,127) ~a:255;  
  Sdlrender.draw_point renderer (120, 120);  
  Sdlrender.draw_point renderer (122, 120);  
  Sdlrender.draw_point renderer (124, 120);  
  Sdlrender.draw_point renderer (126, 120);  
  
  Sdlrender.render_present renderer;  
  Sdltimer.delay 6000;  
  
  Sdl.quit ();  
;;
```

Here are some explanations about how this example works:

- first we have to fill the background with a color, this operation is achieved with two function calls,
the first one, defines the color,

```
Sdlrender.set_draw_color
```

and the second one, requests the renderer to erase the contents of the window, with this selected color.

```
Sdlrender.clear renderer;
```

this is usually what we do, if we want to display, a game, or any other kind of content that we want to animate (in case the background doesn't fill all the window, inside the drawing area).

- After we draw four points with the function below:

```
Sdlrender.draw_point
```

Again we have to define a color first, as if we would choose a color pen:

```
Sdlrender.set_draw_color
```

While painting, the primary colors are: cyan, magenta, and yellow. But with a screen, the primary colors are: red, green, and blue.

Most often these three colors have to be defined in this order. With screen colors, we create a yellow color, by mixing red and green. And leaving blue to zero, or any value close to zero.

In the above example, we select a turquoise color, for the background, with:

```
(0, 255, 255)
```

And a dark-blue, for the four points, with:

```
(0, 0, 127)
```

At the end of all the draw request elements, we should call the renderer with the `render_present` function, to make the drawing visible on screen:

```
Sdlrender.render_present renderer;
```

Then we ask to wait 6 seconds:

```
Sdltimer.delay 6000;
```

This function uses milliseconds, to make pauses smaller than one second (so that we can make smooth animations).

The equivalent of 6 seconds, is 6000 milliseconds.

Then we can quit if we have finished with what we want to display:

```
Sdl.quit ()
```

Sprites Elements

If you want to create a game, or an animation, with real painting, you can take a picture of your painting with a smartphone (if you don't have one, you can ask a friend). Small pieces of images, that are used to display, are often called "sprites" (or "tiles" for background elements.)

The preferred image-file format for use with sdl-2 is the ".bmp" format.

In the console, an easy way to convert a ".png" or ".jpg" image to ".bmp", is to use a command from image-magick:

```
$ convert my_sprite.png my_sprite.bmp
```

Drawing Points

X and Y Coordinates

The positions of the points are given with x and y coordinates along the x and y-axes (abscissa and ordinate.)

```
Sdlrender.draw_point renderer (x, y);
```

The x coordinate is the distance from the left side of the window. The x-axis is the horizontal axis.

The y-axis is the vertical axis. The y coordinate is used reversed, compared to the mathematic direction.

With computers, the y coordinates are often used in the direction: "top-to-bottom".

If you want to place points on the first (top) line of the window, the y coordinate should be: 0

Similarly, the x coordinates for points on the first left column, is also: 0

Considering the width was defined with 320 in the above example, points on the last, right column, should have the x coordinates, set to $320 - 1$, because to have exactly 320 columns, if the first one has coordinate 0

(and not 1), the last one then has coordinate width - 1

```
let width, height = (320, 240)
```

In the programming language used by this package, there is a predefined function, to get the previous integer: `pred`

This is why, to check if a coordinate will be inside the view area of the window, the comparison check has to be: `if x < width` and not: `if x <= width`

Drawing Squares and Rectangles

First Square

You can create rectangles and squares with the "Sdlrect" module.

In the functions: `make1`, `make2`, `make4`, the number represents the arity, which is the number of arguments for a function.

The function `Sdlrect.make` points to the function: `Sdlrect.make2`

The parameters are the same, but are grouped differently with tuples.

Here, `(20, 20)` is the position of the rectangle:

```
let rect = Sdlrect.make (20, 20) (120, 120)
```

And, `(120, 120)` are the dimensions.

Here is a complete example:

```
let () =  
  Sdl.init [`VIDEO];  
  
  let width, height = (320, 240) in  
  
  let window, renderer =  
    Sdlrender.create_window_and_renderer ~width ~height ~flags:[]  
  in  
  
  Sdlrender.set_draw_color renderer ~rgb:(0,255,255) ~a:255;  
  Sdlrender.clear renderer;  
  
  let rect = Sdlrect.make (20, 20) (120, 120) in  
  
  Sdlrender.set_draw_color renderer ~rgb:(0,0,127) ~a:255;  
  Sdlrender.draw_rect renderer rect;  
  
  Sdlrender.render_present renderer;  
  
  Sdltimer.delay 6000;  
  
  Sdl.quit ();  
;;
```

You can test this example with the file "ex_man2.ml", from the "examples/" directory.

Transparency

The parameter `a` from the function to select a color is the opacity:

```
Sdlrender.set_draw_color renderer ~rgb ~a:255;
```

The maximum value (for no transparency) is: 255

The minimum value is: 0 (for completely transparent)

Drawing Lines

```
let () =  
  Sdl.init [`VIDEO];  
  
  let width, height = (320, 240) in  
  
  let window, renderer =  
    Sdlrender.create_window_and_renderer ~width ~height ~flags:[]  
  in  
  
  Sdlrender.set_draw_color renderer ~rgb:(0,255,255) ~a:255;  
  Sdlrender.clear renderer;  
  
  let a = (20, 20) in  
  let b = (120, 120) in  
  
  Sdlrender.set_draw_color renderer (0,0,127) 255;  
  Sdlrender.draw_line renderer (a, b);  
  
  Sdlrender.render_present renderer;  
  
  Sdltimer.delay 6000;  
  Sdl.quit ();  
;;
```

The function below draws a line (a segment of line), from the point `a`, until the second point `b`:

```
Sdlrender.draw_line renderer (a, b);
```

Each point `a` and `b` are defined with 2 coordinates: (x, y)

Filling Squares and Rectangles

And Signatures

We already saw how to draw squares and rectangles, now here is how to also fill the inside area of it.

```
Sdlrender.fill_rect renderer rect;
```

The signature of the function to achieve this is:

```
val fill_rect : t -> Sdlrect.t -> unit
```

It is also a function from the `Sdlrender` module.

The returned element is `unit`, which means that the function doesn't return anything. It produces an effect (side effect), which is to display a filled rectangle on screen.

The type of the second parameter is: `Sdlrect.t`, which is the main type (called `t`) inside the module `Sdlrect`.

In this interface to the `sdl2` library, we separated all the elements into several modules, because there are a lot of elements.

If the library was smaller, with fewer number of elements, we could put everything in a single module, and the type for rectangles could be:

```
type rect
```

This is why a type `Sdlrect.t`, is usually the equivalent for a type that could be named `rect`.

In the signature above the first parameter is `t`, because this function is inside the module `Sdlrender`. This is why this first parameter is equivalent than `Sdlrender.t`, which in a similar way is equivalent than a type that could be named:

```
type renderer
```

Here is the full example:

```
let () =
  Sdl.init [`VIDEO];

  let width, height = (320, 240) in

  let window, renderer =
    Sdlrender.create_window_and_renderer ~width ~height ~flags:[]
  in

  Sdlrender.set_draw_color renderer ~rgb:(0,255,255) ~a:255;
  Sdlrender.clear renderer;

  let rect = Sdlrect.make (20, 20) (120, 120) in

  Sdlrender.set_draw_color renderer ~rgb:(0,0,127) ~a:255;
  Sdlrender.fill_rect renderer rect;

  Sdlrender.render_present renderer;

  Sdltimer.delay 6000;

  Sdl.quit ();
;;
```

This example is exactly the same than the one at step-3 of the manual, except that it calls `Sdlrender.fill_rect`, instead of `Sdlrender.draw_rect`.

Animation Loop

And Recursion

Now that we now how to draw squares and rectangles, we can try to animate it!

A possible solution is to use the function `Sdlrect.move` to move the rectangle at a different position:

```
let rect = Sdlrect.move rect ~x:(new_x_pos) ~y:20 in
```

In the example below, we keep the same y coordinate, this is why the rectangle will move along the y-axis.

A possibility to display several frames, in order to achieve an animation is to create a loop function which will repeat itself several times, for each image of the animation.

This loop function repeats itself several times, this is why it should be created with the `rec` keyword after the `let` statement.

This loop function repeats itself several times, by calling itself at the end of its operations.

In a function that produces a display, typically the first operation is to erase the previous frame image, by for example filling all the drawing area with a selected color.

In the example below, this is achieved with the two function calls:

```
Sdlrender.set_draw_color renderer ~rgb:(0,255,255) ~a:255;  
Sdlrender.clear renderer;
```

In the loop function, we put an iterator variable `i`, and we also pass the initial rectangle `rect`, which will be modified at each steps of the loop function, with `Sdlrect.move`:

```
let () =  
  Sdl.init [`VIDEO];  
  
  let width, height = (320, 240) in  
  
  let window, renderer =  
    Sdlrender.create_window_and_renderer ~width ~height ~flags:[]  
  in  
  
  let rect = Sdlrect.make (20, 20) (60, 60) in  
  
  let rec loop i rect =  
    Sdlrender.set_draw_color renderer ~rgb:(0,255,255) ~a:255;  
    Sdlrender.clear renderer;  
  
    Sdlrender.set_draw_color renderer ~rgb:(0,0,127) ~a:255;  
    Sdlrender.fill_rect renderer rect;  
  
    Sdlrender.render_present renderer;  
    Sdltimer.delay 120;  
  
    let rect = Sdlrect.move rect ~x:(20 + i) ~y:20 in  
  
    if i < 80 then loop (succ i) rect  
  in  
  loop 0 rect;  
  
  Sdl.quit ();  
;;
```

You can find this example ready to run as the file `ex_man7.ml`, in the `examples/` directory:

```
$ ocaml -I src/ sdl2.cma examples/ex_man7.ml
```

A Better Organisation of the Code

The "MVC" organisation

If you want to continue with the previous example to create something more complex, with more elements, we will need to use a better organisation of the code, than to put all the code in the same function for the main loop.

In the computer-science area, the organisation of code recommended for programs which present a window to a user, is the organisation of code called "MVC".

It means: "Model-View-Controller"

We have to separate each of these three elements, so that we can grow our software with a code correctly organised.

The "Model" contains, the data which is represented on screen (or used by the program to do something.) In our example this is the rectangle, and the background color.

In the programming language we are using, the type of this data also has to be defined first, for something constructed like that. Here we can use this type:

```
type game_state = {  
  bg_color: int * int * int;  
  rect: Sdlrect.t;  
}
```

This type is then initialised like this:

```
let rect = Sdlrect.make (20, 20) (60, 60) in  
  
let game_state = {  
  bg_color = (0, 255, 255);  
  rect = rect;  
} in
```

The code to display one state of the data:

```
let display game_state =  
  let bg_color = game_state.bg_color in  
  let rect = game_state.rect in  
  
  Sdlrender.set_draw_color renderer ~rgb:bg_color ~a:255;  
  Sdlrender.clear renderer;  
  
  Sdlrender.set_draw_color renderer ~rgb:(0,0,127) ~a:255;  
  Sdlrender.fill_rect renderer rect;  
  
  Sdlrender.render_present renderer;  
  ()  
in
```

This display function is the "View".

And the main-loop then becomes:

```
let rec loop i game_state =  
  display game_state;  
  let game_state = update i game_state in  
  Sdltimer.delay 120;  
  
  if i < 80 then loop (succ i) game_state  
in
```

And we group the functions that update the data together in a function which is also called once at each animation step:

```
let update i game_state =
  let rect = game_state.rect in
  let rect = Sdlrect.move rect ~x:(20 + i) ~y:20 in

  { game_state with
    rect = rect;
  }
in
```

This update function is: the "Controller".

Please find the result in the file `examples/ex_man8.ml`, that you can run as:

```
$ ocaml -I src/ sdl2.cma examples/ex_man8.ml
```

With this architecture, it becomes easier to develop our code further.

Continuing with the "MVC" model

Populating the area with several Entities

With the separation of code realised at step-8 of this "manual", we can now add more entities in our small-demo.

In the example `ex_man9.ml`, with transform our previous `rect` into an entity, initialised with:

- `new_rect ()`

We add 12 of these entities in our `game_state`, with:

- `List.init 12`

These entities now have a direction, with the field: `dir`

This direction is used in the function `update_rect`, to move the entities.

The function `update_rect`, also changes this direction to the opposite direction when an entity reaches the side of their living area.

(with the code below)

```
let dir = if x < 0 then d0 else dir in
let dir = if y < 0 then d1 else dir in
let dir = if x + 20 > width then d2 else dir in
let dir = if y + 20 > height then d3 else dir in
```

Check the result with this command:

```
$ ocaml -I src/ sdl2.cma examples/ex_man9.ml
```

!!! Warning: The cross of the window handle is not connected yet with an exit in the demo, so you'll have to wait: (600 * 60) milliseconds, (= 36000 milliseconds), or 36 seconds, for this demo to exit.

Processing events, (and the "escap" key-press event) will be seen in the next step (step-10) of these manuals.

Processing Events

The Events Loop

Processing events can be done with the `Sdlevent.poll_event` function. An event can be a key-press from the user, or a move of the mouse, or a button-click on the button of the mouse.

Pressing the little cross on the handle of the window, is also an event that we should connect with an exit, if we want that the user can exit while pressing this cross.

If we make a small game, or any other kind of application, getting only one event at each game-step (or frame) is not enough. Because there might be several events arriving at each frame, for example if the user is moving the mouse, the event queue will possibly contain several, or even a lot of events.

This is why we have to create a loop to get all the events repeatedly at each frame.

A possible way to achieve this process is to create a recursive function that will loop until the function `Sdlevent.poll_event ()` returns `Some event`, and stop by returning to the caller function, when we receive `None`.

```
val poll_event : unit -> t option
```

This function belongs to the module: `Sdlevent`, and can be called like that:

```
match Sdlevent.poll_event () with
| Some event -> (* event-processing-loop *)
| None -> (* returning *)
```

You can run and study the example: `ex_man10.ml`, from the directory: `examples/`. This example, starts back, from the `ex_man8.ml` example, and adds events handling.

Press keys 'A' and 'B', to change the background color:

```
$ ocaml -I src/ sdl2.cma examples/ex_man10.ml
```

You can check, that now, the cross of the window handle is connected correctly with an exit.

Loading Sprites

Check example: `ex_man11.ml`, for an example of use of an image loaded with pieces of it used as sprites:

```
$ ocaml -I src/ sdl2.cma examples/ex_man11.ml
```

PS: Be careful, the image should not be an indexed one if you want to use a color-key for transparency.
